N. Thyjasree
192524438.

Date:- 29/08/26

C05_AT3

Flow chart to Coding

1. Convert a flowchart of Depth-First Search (DFS) using a stack into a C program.

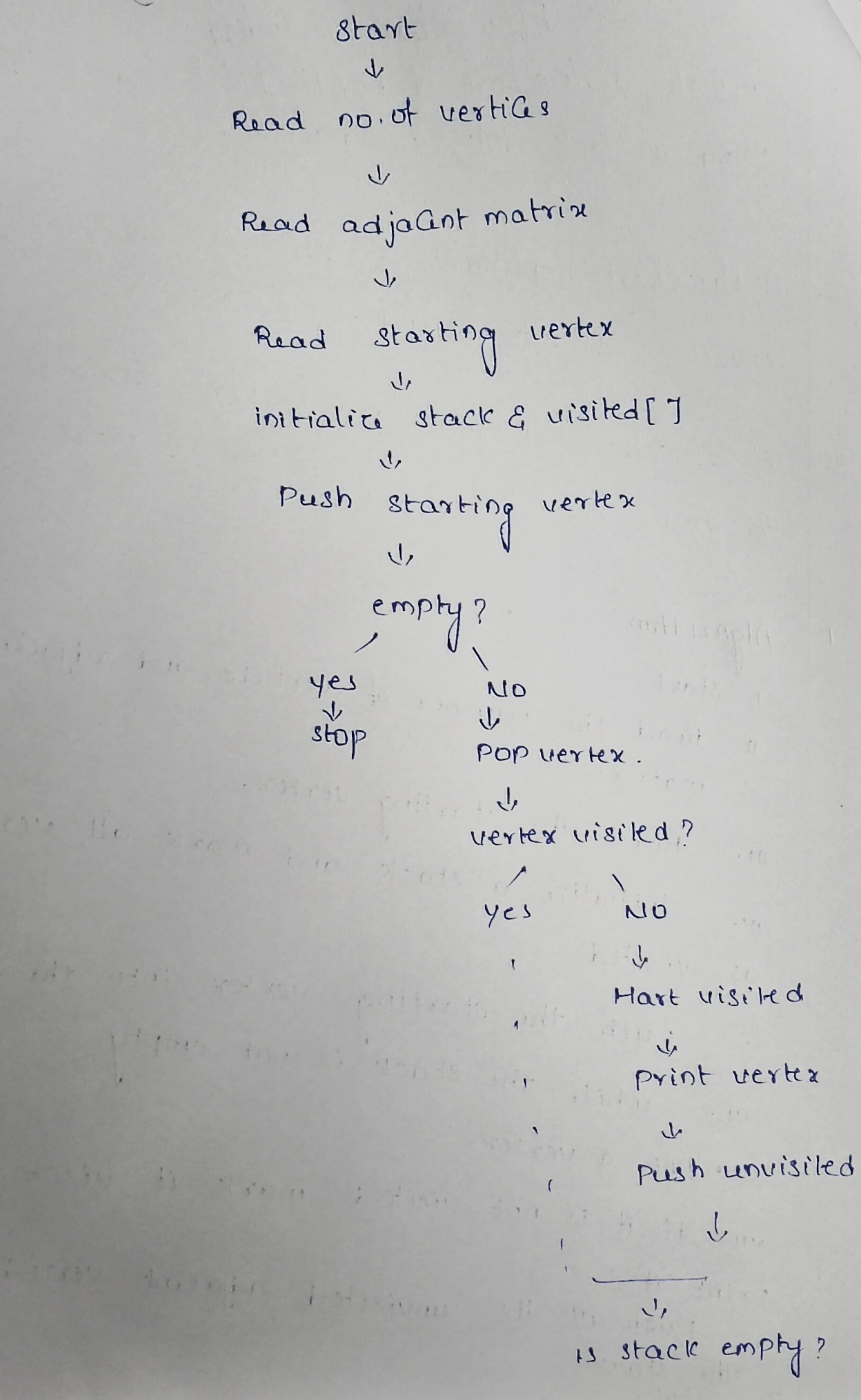
2. A flowchart implements a Dijkstra Algorithm and Converts it into C Code.

3. A flowchart performs prims Algorithm. Convert the flowchart into a recursive C program.

1. Algorithm:

i. Start
ii. Read the number of vertices and adjacency matrix.
iii. Read the starting vertex.
iv. initialize a stack and mark all vertices as visited.
v. push the starting vertex into the stack.
vi. while the stack is not empty:
- pop a vertex.
vii. if it is not visited, mark it visited and print it.
- Push all its unvisited adjacent vertices into the stack
7. Stop.

Flowchart


Start
Read no. of vertices
Read adjacent matrix
Read starting vertex
initialize stack & visited[ ]
Push starting vertex
empty?
yes
stop
NO
POP vertex.
vertex visited?
yes
NO
Mark visited
print vertex
Push unvisited
is stack empty?

```
#include <stdio.h>

int main()

{
    int graph[10][10], stack[10],
  visited[10] = {0};
        int n, start, top=-1;
        int i, j, v;
        printf("Enter number of vertices:");
        scanf("%d", &n);
        printf("Enter adjacency matrix:\n");
        for(i=0; i<n; i++)
            for(j=0; j<n; j++)
                scanf("%d", &graph[i][j]);
        printf("Enter starting vertex:");
        scanf("%d", &start);
        stack[++top]= start-1;
        printf("DFS:");
        while(top>=0)
        {
            v= stack[top--];
            if(visited[v]==0)
            {
                printf("%d", v+1);
                visited[v]=1;
            for(i=n-1; i>=0; i--)
```

```
{
  if (graph [v][i] == 1 && visited [i] == 0)
        stack [++top] = i;
    }
  }
}
return 0;
}
```

input

vertices = 4

matrix

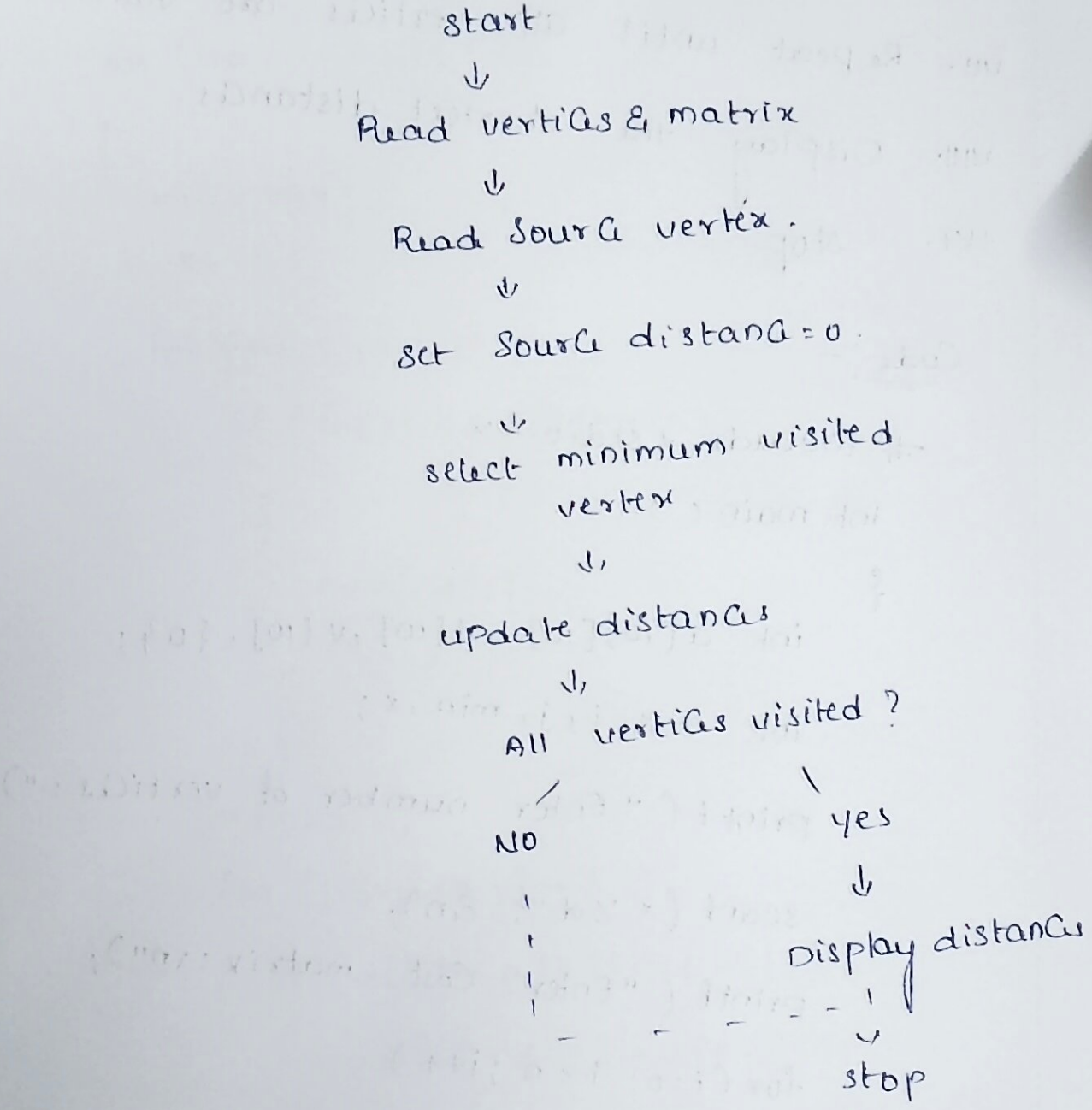
0 1 1 0
1 0 0 1
1 0 0 1
0 1 1 0

Starting vertex : 1

output

DFS : 1 2 4 3.

Flowchart



Algorithm

I. start

II. Read the number of vertices and Cost matrix

III. Read the source vertex.

IV. set the source distance as 0.

V. select the unvisited vertex with minimum distance.

vii. update the distances of its adjacent vertices.

viii. Repeat until all vertices are visited.

ix. Display the shortest distances.

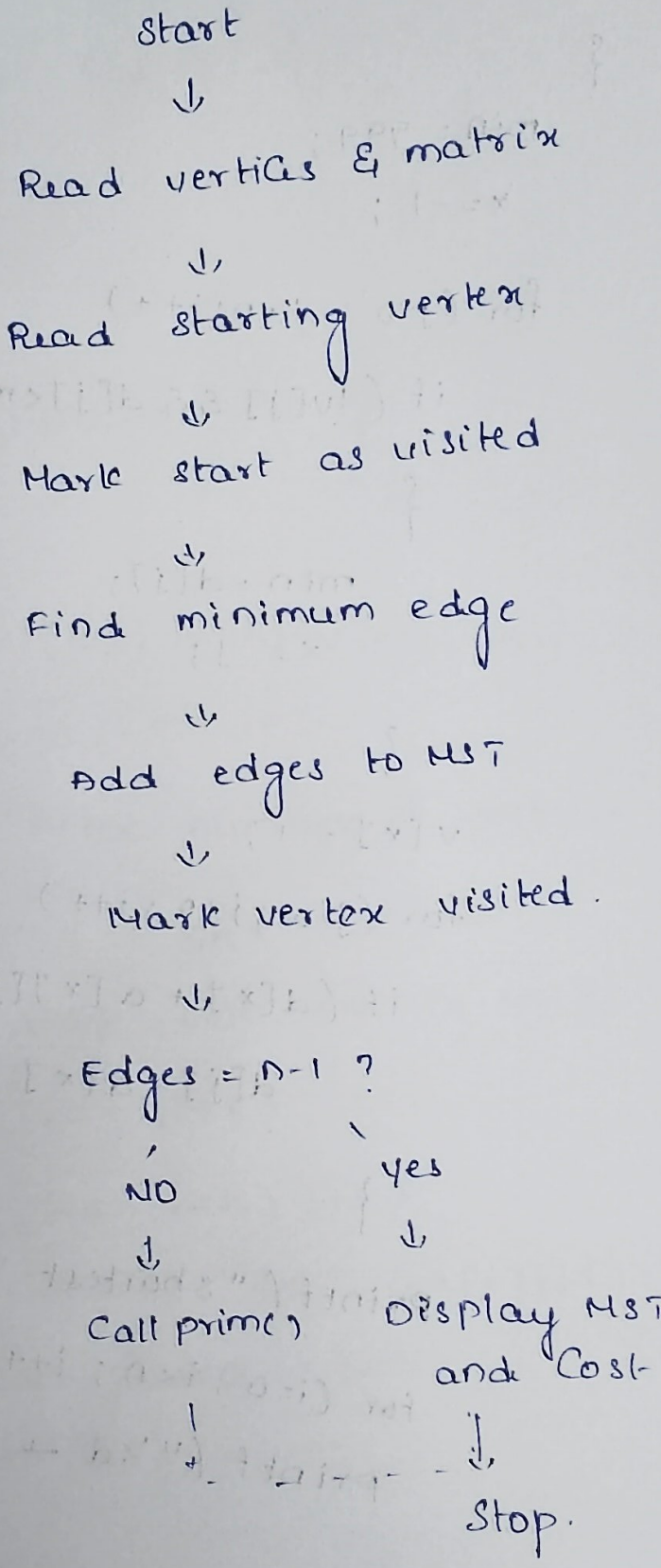
x. Stop.

Code :-

```
#include <stdio.h>
int main()
{
    int a[10][10], d[10], v[10] = {0};
    int n, s, i, j, min, x;
    printf("Enter number of vertices : ");
    scanf("%d", &n);
    printf("Enter Cost matrix :\n");
    for(i=0; i<n; i++)
        for(j=0; j<n; j++)
            scanf("%d", &a[i][j]);
    printf("Enter Source : ");
    scanf("%d", &s);
    s--;
    for(i=0; i<n; i++)
        d[i] = a[s][i];
```

```
    d[s]=0;
  for (i=0; i<n; i++)
  {
    min=999;
     x=-1;
    for (j=0; j<n; j++)
       if (!v[j] && d[j]<min)
        {
           min=d[j];
            x=j;
        }
      v[x]=1;
      for (j=0; j<n; j++)
        if (d[x]+a[x][j]<d[j])
            d[j]=d[x]+a[x][j];
        }
      printf("shortest distances:\n");
       for (i=0; i<n; i++)
        printf("%d -> %d = %d\n", s+1, i+1
                                  ,d[i]);
        return 0;
    }.
```

3. Flowchart


Start
Read vertices & matrix
Read starting vertex
Mark start as visited
Find minimum edge
Add edges to MST
Mark vertex visited.
Edges = n-1 ?
NO
yes
Call prim()
Display MST and Cost.
Stop.

Algorithm:

I. Start

II. Read the number of vertices and Cost matrix.

III. Read the starting vertex.

IV. Mark the starting vertex as visited

V. Find the minimum edge Connecting visited and unvisited vertices.

VI. Add the edge to the Minimum spanning tree.

VII. Mark the new vertex as visited.

VIII. Recursively repeat until n-1 edges are selected.

IX. Display the total Cost.

X. Stop

Code :-

```
#include <stdio.h>
int a[10][10], visited[10];
int n, edges=0, Cost=0;
void prim();
{
  int i, j, min=999, x=-1, y=-1;
  if(edges==n-1)
    return;
  for(i=0; i<n; i++)
    if(visited[i])
      for(visited[i])
        if(!visited[j] && a[i][j]!=0 &&
          a[i][j]<min)
        {
          min=a[i][j];
          x=i;
          y=j;
        }
printf("%d - %d = %d\n", x+1, y+1, min);
Cost += min;
visited[y]=1;
edges++;
```

```
    prim( );

}
int main( )
{
 int i, j, start;
  printf("Enter number of vertices: ");
  scanf("%d", &n);
  printf("Enter Cost matrix: \n");
   for (i=0; i<n; i++)
        for (j=0; j<n; j++)
            scanf("%d", &a[i][j]);

  Printf("Enter starting vertex: ");
  Scanf("%d", &start);
  visited [start-1]=1;
  printf("minimum spanning tree: \n");
  prim( );
   return 0;
  }
```